PhishMind

AI-Powered Phishing Email Detector — Full Project Reference

This document is a complete technical reference for the PhishMind project: what it is, how its backend, frontend and Chrome extension work together, its detection and scoring logic, its data model, its configuration, and the bugs already found and fixed during development. It is written so it can be pasted directly into any AI assistant to give it full working context on this project before asking it questions or requesting changes.

Stack: Python / FastAPI backend · SQLite · Google Gemini AI · React + TypeScript + Vite + Tailwind dashboard · Chrome Manifest V3 extension.

1. Overview

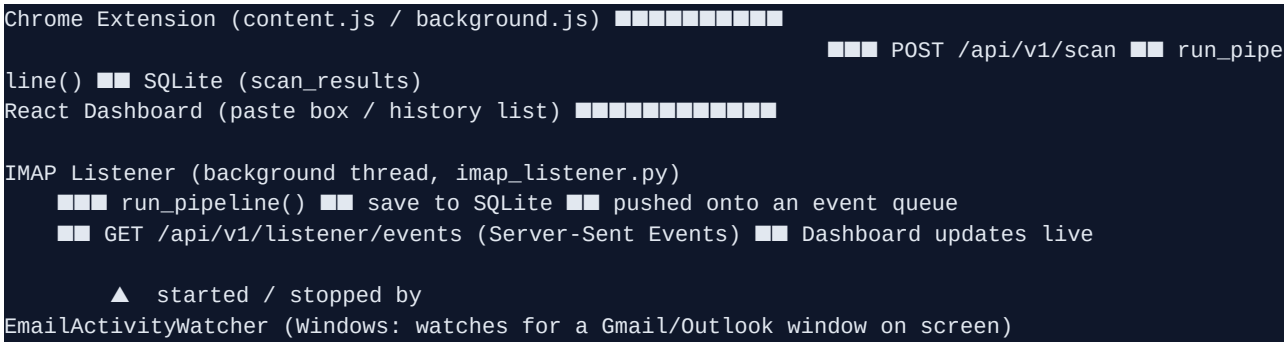
PhishMind is an AI-assisted phishing email detector made of three coordinated pieces that all talk to one shared backend:

- **Backend (Python / FastAPI):** parses a raw email (.eml file or pasted text), runs deterministic technical and URL heuristics, calls Google Gemini for semantic and social-engineering analysis, verifies the claimed sender brand against the actual sending domain, and deterministically combines everything into a 0–100 risk score with a SAFE / SUSPICIOUS / PHISHING verdict. It can also passively watch a live Gmail/IMAP inbox in the background and push new results out in real time.
- **Dashboard (React + TypeScript + Vite + Tailwind):** a 3-pane web app — scan history and a paste-to-scan box on the left, the full email with a warning banner in the middle, and a score/evidence inspector panel on the right. It updates live via Server-Sent Events (SSE) whenever the background IMAP listener scans new mail.
- **Chrome extension (Manifest V3):** reads the email currently open in Gmail's own web interface, submits it to the same backend, and overlays a small floating verdict badge directly on the Gmail page — so a person gets a warning without ever leaving their inbox.

Core design principle (stated explicitly in the backend's own README): the AI model never invents the final score. Gemini only supplies structured, itemized evidence — the sender's likely intent, which manipulation techniques appear, a semantic risk hint, translations for non-English mail. A single deterministic module (`app/risk/scorer.py`) is the only place the final 0–100 number and verdict are computed, from that evidence plus separate rule-based technical/URL checks. That keeps results reproducible and stops the AI from being able to hallucinate a score.

2. Architecture

All three entry points funnel into one shared scan pipeline and one shared database, so a result looks identical no matter which surface triggered the scan:



`app/pipeline.py`'s `run_pipeline(email_data)` function is the single shared code path called both by the manual `/api/v1/scan` endpoint and by the background IMAP listener, guaranteeing identical scoring behavior no matter which surface triggered the scan.

3. Backend Details (main.py)

3.1 API Endpoints

Method	Path	Purpose
GET	/api/v1/health	Liveness check, returns status ok.
POST	/api/v1/scan	Body: raw_eml_base64 or raw_email_text (exactly one). Parses the email, runs the full detection pipeline, saves the result, returns it with a new scan id. Returns a clear error if the email can't be parsed.
GET	/api/v1/history?limit=50	Recent scans, newest first. Limit is capped between 1 and 500.
GET	/api/v1/scan/{scan_id}	Fetch one previously stored scan by id.
POST	/api/v1/scan/{scan_id}/feedback	Body: feedback: 'spam' or 'valid'. Used by the 'Spam or Valid?' prompt shown after a risky verdict, in both the extension and the dashboard.
POST	/api/v1/listener/start	Body: host, username, app_password. Starts the background IMAP listener for that mailbox. Returns an error if a listener is already running.
POST	/api/v1/listener/stop	Stops the background IMAP listener.
GET	/api/v1/listener/status	Whether the IMAP listener is currently running, and its last error if any.
GET	/api/v1/watcher/status	Whether the Windows email-activity watcher is enabled and currently sees an email window open.
GET	/api/v1/listener/events	Server-Sent Events stream. Sends a scan_result event the instant the IMAP listener finds and scores new mail, plus a lightweight keepalive ping every second when idle.

CORS is configured to allow the origins listed in `ALLOWED_ORIGINS`, plus (as a safety net) any `localhost` or `127.0.0.1` port at all — so the dashboard keeps working even if Vite's dev server auto-bumps to a different port.

3.2 Startup Wiring

- On startup, the database is created/migrated if needed.
- If no Gemini API key is configured, the app logs a warning but keeps running — scans just fall back to technical + URL heuristics only, with a clear explanation that AI analysis was unavailable.
- If IMAP credentials (host, username, app password) aren't all set in .env, no background listener or watcher starts at all — the app only responds to manual scans.
- Otherwise, if `EMAIL_WATCH_ENABLED` is true (the default): the IMAP listener only runs while a Gmail/Outlook/mail window is visibly open on screen, saving resources when the user isn't near their inbox. If false: the IMAP listener runs continuously, 24/7, regardless of what's on screen.

3.3 Live updates (Server-Sent Events)

Each new scan result found by the background IMAP listener is pushed onto a thread-safe queue. The `/api/v1/listener/events` endpoint drains that queue continuously and streams each result to the dashboard the instant it's ready, so new mail shows up in the UI in real time without the dashboard needing to poll.

3.4 IMAP Listener ↔ Window-Activity Watcher

On Windows, `EmailActivityWatcher` checks every few seconds (default 5s) whether any visible window's title contains a mail-related keyword (default: gmail, outlook, mail). When one first appears, it starts the IMAP listener; once no such window has been visible for a grace period (default 30s, to tolerate brief tab switches), it stops the listener. This means the live IMAP connection to Gmail only exists while the user is actually looking at their mail — it isn't running in the background all day by default.

3.5 Detection Pipeline (pipeline.py)

Every scan, regardless of source, runs these steps in order:

- **1. Technical checks** — SPF/DKIM/DMARC authentication results, Reply-To vs. sender domain mismatch, lookalike link domains. (0–25 points)
- **2. URL analysis** — every link in the email is checked for suspicious patterns. (0–15 points)
- **3. Gemini AI analysis** — the model reads the subject, sender, body and links, and returns structured evidence: likely intent, manipulation techniques used, suspicious evidence, a semantic risk hint, and (if the email isn't in English) a full translation. If Gemini is unavailable or fails, a safe degraded fallback result is used instead and the scan still completes.
- **4. Brand verification** — if the email claims to be from a known brand (PayPal, Microsoft, a bank, etc.), the actual sending domain is checked against that brand's real domain.
- **5. Final scoring** — all of the above is combined deterministically into the final 0–100 risk score and verdict.
- **6. Evidence assembly** — every red flag found in steps 1–4 is gathered into one plain-English evidence list shown to the reader.

4. Scoring Model (app/risk/scorer.py)

The final risk score is the sum of five independently-capped categories, always adding up to at most 100:

Category	Max points	What it measures
Technical	25	Failed/soft-failed SPF, DKIM or DMARC checks; a Reply-To address that doesn't match the sender; links pointing to a different domain than the sender.
Intent	30	How dangerous the email's apparent goal is (e.g. stealing credentials, financial fraud and malware delivery score highest; simple information requests score lower), scaled by how confident the AI is.
Manipulation tactics (social engineering)	30	Points added per detected persuasion technique (urgency, authority, fear, etc.), weighted by how severe each one is.
Suspicious links	15	Rule-based scoring of every URL: not using HTTPS, raw IP addresses instead of domains, unusually long or obfuscated links, suspicious keywords like ‘verify’ or ‘confirm’ in the URL, throwaway top-level domains.
Does the story add up? (semantic support)	10	A bonus derived from the AI's own overall semantic-risk read of the email as a whole.

Verdict bands: 70 or higher → PHISHING. 30–69 → SUSPICIOUS. Below 30 → SAFE.

4.1 Technical checks in detail

- SPF fail adds 10 points, SPF softfail adds 5. DKIM fail adds 8. DMARC fail adds 7.
- A Reply-To domain that doesn't match the sender's own domain adds 10 — a classic sign of credential-harvesting, where the visible sender looks legitimate but replies get routed somewhere else.
- If any link in the email points to a different registered domain than the sender's own domain, that's flagged as suspicious evidence.
- The technical score is capped at 25 no matter how many checks fail.

4.2 URL checks in detail

Every link is checked for: whether it uses HTTPS, whether it's a raw IP address instead of a domain name, whether it's unusually long (a common obfuscation tactic), whether it contains suspicious keywords in the path (login, verify, account, secure, update, confirm, signin, password, banking), and whether it uses a top-level domain often abused for throwaway phishing sites (.zip, .mov, .xyz, .top, .click, .gq, .tk, .cf, .ml). Each hit adds 3 points, capped at 15 total, and is written up as a plain-English evidence line (e.g. “Unusually long URL (possible obfuscation)”).

5. Database (SQLite)

Everything is stored in a single local SQLite file (default name `phishmind.db`), in one table, `scan_results`:

```
id                      auto-incrementing scan id
timestamp               when the scan happened
subject, sender, body   the email itself
claimed_brand           brand the email claims to be from, if any
is_brand_verified       whether that claim checked out against the real domain
risk_score              final 0-100 score
verdict                 SAFE / SUSPICIOUS / PHISHING
breakdown_json          the five category sub-scores, as JSON
social_engineering_json detected manipulation techniques, as JSON
suspicious_evidence_json the plain-English evidence list, as JSON
explanation              one-paragraph human explanation of the verdict
actionable_advice_json  what the reader should do next, as JSON
source                  'manual' (pasted/extension) or the live listener
user_feedback           'spam' or 'valid' - what the reader said afterwards
```

The `user_feedback` column records what the reader actually decided (spam or valid) after seeing a risky verdict, purely for later comparison against PhishMind's own call — it isn't used to retrain anything in this version of the app.

6. Gemini AI Integration

PhishMind calls Google's Gemini model (default: `gemini-flash-lite-latest`) with a strict structured-output schema, so the response always comes back as clean, predictable JSON rather than free text that needs to be parsed. The model is asked to return:

- **intent** — one of 11 categories: credential theft, financial fraud, data theft, malware delivery, OAuth abuse, MFA manipulation, information disclosure, account verification, impersonation, benign, or unknown — plus a confidence score from 0 to 1.
- **claimed_brand** and **requested_action** — what brand the email pretends to be, and what it's actually asking the reader to do.
- **social_engineering** — a list of manipulation techniques found, each with supporting evidence and a severity (low/medium/high).
- **suspicious_evidence** and **ai_assisted_indicators** — plain-English red flags, and stylistic hints the text may be AI-generated (never stated as a certainty).
- **semantic_risk** — the model's own overall 0–100 risk read of the email.
- **explanation** and **user_advice** — a plain-English summary and concrete next steps for the reader.
- **detected_language, translated_subject, translated_body** — if the email isn't in English, the model detects the language and provides a full English translation so scoring and display both work regardless of the original language.

The model is explicitly instructed to reason only from observable evidence, stay conservative about flagging techniques, never claim certainty that text was AI-generated, and always write its explanation and advice in English even when translating a non-English email. If the Gemini API key isn't configured, or the call fails for any reason (network issue, quota, bad response), the analyzer never lets that crash a scan — it silently falls back to a neutral result (unknown intent, zero semantic risk) with a clear note that AI analysis wasn't available, and the scan still

completes using the technical and URL checks alone.

7. Email Parsing

Raw emails (either a pasted .eml file, or scraped text from Gmail's own page) are parsed into a subject, sender address, reply-to address, body text and a list of links. If an email only has an HTML body, that HTML is converted into readable text and its links are extracted separately, so nothing downstream ever runs on an empty body. Authentication headers (SPF/DKIM/DMARC results) are captured here and interpreted later by the technical scoring step.

8. Brand / Sender Verification

If an email claims to be from a recognizable brand (Netflix, Microsoft, Google, Apple, Amazon, PayPal, a major bank, Facebook, Instagram, LinkedIn, Dropbox, DocuSign, Adobe, YouTube, Outlook, Twitter/X, Spotify, Steam, Discord, WhatsApp, and a few others), PhishMind checks the actual sending domain against that brand's real, known domain. If it claims an unrecognized brand, a more conservative check is used instead. Two things are always kept separate and never confused: “we couldn't verify this” (e.g. the email is missing sender information) is reported differently from “we checked, and it's spoofed” — so a missing header is never mistaken for proof of fraud.

9. Desktop Notifications

On Windows, when the background IMAP listener scores a live email as SUSPICIOUS or PHISHING, PhishMind fires a native desktop toast notification (via the `win10toast` library) showing the subject, sender and risk score — so the reader gets a heads-up even if the dashboard isn't open. This is entirely optional (`DESKTOP_NOTIFICATIONS` in config) and any failure to show a notification is silently ignored rather than breaking the scan.

10. Configuration Reference (backend/.env)

Key	Purpose
GEMINI_API_KEY	Google Gemini API key. Left blank, AI analysis is skipped gracefully.
GEMINI_MODEL	Which Gemini model to call. Default: gemini-flash-lite-latest.
DATABASE_PATH	Path to the SQLite database file. Default: phishmind.db.
ALLOWED_ORIGINS	Comma-separated list of frontend URLs allowed to call the API.
HOST / PORT	Address the backend server listens on. Default: 0.0.0.0:8000.
IMAP_HOST	The mailbox's IMAP server. Default: imap.gmail.com.
IMAP_USERNAME	Mailbox address to auto-watch. Left blank, auto-start is disabled.
IMAP_APP_PASSWORD	A Gmail App Password (not the account's normal password).
DESKTOP_NOTIFICATIONS	true/false — native toast alerts for risky live emails. Default: true.
EMAIL_WATCH_ENABLED	true/false — only scan while a mail window is visibly open (true), or scan continuously (false). Default: true.
EMAIL_WATCH_KEYWORDS	Window-title keywords that count as “mail is open”. Default: gmail, outlook, mail.
EMAIL_WATCH_POLL_SECONDS	How often to check for a visible mail window. Default: 5.
EMAIL_WATCH_GRACE_SECONDS	How long a mail window can be hidden before scanning pauses. Default: 30.

10.1 Backend dependencies

fastapi, uvicorn, pydantic, python-dotenv, google-genai, tldextract, imapclient, beautifulsoup4, sse-starlette, python-multipart, pytest, httpx, and win10toast (Windows only).

11. Frontend Dashboard

Built with React, TypeScript, Vite and Tailwind CSS. Laid out as three panes:

- **Left sidebar** — a box to paste a raw email and check it on demand, a “Connect inbox” button that opens a form to link a live IMAP mailbox, and the running history of everything scanned so far.
- **Center pane** — the full email, with a colored banner across the top for anything SUSPICIOUS or PHISHING, plus small badges noting whether a claimed brand checked out and whether the email was auto-translated.
- **Right pane (inspector)** — a circular 0–100 score ring, a plain-language verdict, a breakdown of the five scoring categories, the specific evidence found, and concrete next-step advice.

The dashboard subscribes to the backend's live event stream, so anything the background IMAP listener scans appears instantly without needing to refresh the page.

Current visual style: a clean white theme with muted, simple colors (white background, very light gray panels, slate text, and calm red/amber/green for danger/caution/safe) and plain-language copy throughout (e.g. “This looks like a scam” instead of raw jargon like “Warning: SUSPICIOUS (Score: 69)”), rather than a dark, terminal-style security console.

12. Chrome Extension

A Manifest V3 extension that works directly inside Gmail's own web page (mail.google.com):

- When an email is opened in Gmail, the extension reads its subject, sender and body directly from the page, builds an email-like message from it, and sends it to the same backend used by the dashboard.
- The result appears as a small floating badge in the corner of the page — a colored circle with the risk score. Clicking it expands into a full card with the verdict, explanation, evidence and advice, plus a quick “Spam or Valid?” feedback prompt for risky emails.
- For SUSPICIOUS or PHISHING results, a native browser notification also fires.
- The badge correctly disappears when no email is open (e.g. back on the inbox list), and refreshes cleanly for each new email opened — a bug where the previous verdict could linger on screen has been fixed.
- The toolbar popup shows connection status to the backend, the most recent scan, buttons to re-check the currently open email or open the full dashboard, and settings for the backend/dashboard URLs and whether to auto-scan and show the feedback prompt.
- Because Gmail's page doesn't expose raw email headers, the extension can't check SPF/DKIM/DMARC the way the backend's direct .eml parsing or live IMAP listener can — so its technical sub-score is naturally lower. Full header-based scoring happens via the dashboard's paste box or the live inbox connection.

13. Known Issues Already Found & Fixed

These are real bugs already diagnosed and resolved during development, kept here so they aren't accidentally reintroduced:

- **Gmail IMAP connections don't self-refresh.** Gmail's IMAP server doesn't notice new mail on an already-open connection unless the client explicitly asks it to refresh (a NOOP command). Without this, only the very first email after each restart was ever detected. Fixed by sending a refresh command on every polling tick.
- **Reconnects used to lose track of what had already been scanned.** The listener now remembers the last email it handled across a dropped/reconnected connection, so a brief network blip can't cause it to silently skip mail that arrived during the outage, and can't cause it to re-scan the whole backlog either.
- **The one-time catch-up scan of existing mail now only runs once**, on the very first connection — not on every reconnect.
- **Reconnect attempts now back off** (3 seconds, doubling up to a 60-second cap) instead of retrying in a tight loop.
- **The Chrome extension's floating badge used to get "stuck."** It would keep showing the last verdict even after the reader left that email or opened a completely different one. Fixed so the badge clears the moment no email is open, and always reflects only the currently open email.
- **A URL-cleanup bug** that could accidentally cut real characters off the end of a legitimate link has been corrected to only strip genuine trailing punctuation.
- **CORS was previously tied to one fixed dashboard URL**, so the dashboard would show "Failed to fetch" whenever its dev server picked a different port; the backend now accepts any localhost port automatically.
- **The dashboard's original look** (a dark, terminal-style security console with raw jargon like THREAT SCORE and SUSPICIOUS (Score: 69)) has been replaced with a plain-language, white/light-themed design meant to feel approachable to a non-technical reader.

This document was generated as a project reference for PhishMind. No API keys, passwords or other secret values are included anywhere in this document — only configuration key names and what they control.